

Pesquisa de bibliotecas para o firmware do Micromouse

1. Introdução

Este relatório apresenta e justifica as bibliotecas inicialmente selecionadas para o desenvolvimento do firmware do robô Micromouse do projeto PI1 2026/1. O projeto utiliza o microcontrolador ESP32-S3-DEV-KIT-N16R8, programado no VSCode com a extensão PlatformIO e o framework Arduino (espressif32).

As escolhas levam em consideração compatibilidade com o ESP32-S3, leveza, suporte a recursos de hardware nativos do chip e maturidade das bibliotecas na comunidade de robótica.

2. Sensores de Distância — VL53L0X ×3

2.1 Biblioteca escolhida

Campo	Valor
Biblioteca	pololu/VL53L0X
PlatformIO ID	1804
Linguagem	C++
Dependências	Wire.h (nativa do Arduino)

Repositório / Documentação: <https://github.com/pololu/vl53l0x-arduino>

2.2 Justificativa

O projeto emprega três sensores VL53L0X no mesmo barramento I2C. Como todos compartilham o endereço padrão 0x29, é necessário inicializá-los sequencialmente via pino XSHUT, atribuindo endereços distintos a cada um.

Motivos da escolha:

- API enxuta e direta, sem camada HAL adicional.
- Amplamente usada em projetos Micromouse e de robótica competitiva.
- Exemplos documentados para múltiplos sensores no mesmo barramento.
- Expõe configurações de baixo nível (timeout, modo de precisão, timing de medição) úteis para ajuste do loop de controle.
- Nenhuma dependência além do Wire.h nativo, mantendo o firmware leve.

2.3 Alternativa considerada

A biblioteca adafruit/Adafruit_VL53L0X também é funcional, porém exige as dependências Adafruit BusIO e Adafruit Unified Sensor, adicionando peso adicional. Só seria interessante caso outros componentes do projeto já utilizassem o ecossistema Adafruit.

Alternativa (Adafruit): https://github.com/adafruit/Adafruit_VL53L0X

3. Sensor Inercial — MPU-6050

3.1 Biblioteca escolhida

Campo	Valor
Biblioteca	ElectronicCats/MPU6050
Linguagem	C++
Dependências	Wire.h (nativa do Arduino)
Diferencial	Suporte ao DMP (Digital Motion Processor)

Repositório / Documentação: <https://github.com/ElectronicCats/mpu6050>

3.2 Justificativa

O MPU-6050 combina acelerômetro e giroscópio de 3 eixos no mesmo encapsulamento e se comunica via I2C. O fork da ElectronicCats é o mais ativamente mantido e inclui suporte ao DMP interno do chip.

Motivos da escolha:

- DMP realiza fusão de sensores internamente, reduzindo a carga de processamento no ESP32.
- Compatível com ESP32-S3 via Wire.h sem adaptações.
- Comunidade ativa e issues resolvidos com frequência.

3.3 Alternativa considerada

A adafruit/Adafruit_MPU6050 é mais simples e adequada para leituras diretas sem uso do DMP. Recomendada para prototipagem rápida, mas sem o ganho de eficiência do processamento embarcado no chip.

Alternativa (Adafruit): https://github.com/adafruit/Adafruit_MPU6050

4. Ponte H — L298N

4.1 Biblioteca escolhida

Campo	Valor
Biblioteca	Nenhuma — controle direto via API nativa
PWM	LEDC (periférico nativo do ESP32)
Direção	digitalWrite() — API nativa do Arduino
Dependências	Nenhuma

Documentação LEDC (ESP32 Arduino): <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html>

4.2 Justificativa

O L298N é controlado por sinais digitais de direção e um sinal PWM de velocidade, não exigindo protocolo de comunicação serial. O ESP32-S3 gera PWM pelo periférico LEDC (LED Control) com até 8 canais independentes. A função analogWrite() padrão do Arduino não mapeia de forma confiável nos canais LEDC do ESP32-S3, portanto a API LEDC é usada diretamente:

```
ledcSetup(canal, frequencia, resolucao);  
ledcAttachPin(pino, canal);  
ledcWrite(canal, duty_cycle);
```

Motivos para não adotar biblioteca externa:

- O controle do L298N é simples: dois digitais e um PWM por motor.
- Bibliotecas de terceiros para L298N foram projetadas para Arduino Uno/Mega e não garantem suporte ao LEDC do ESP32-S3.
- Implementação própria em classe C++ oferece controle total e sem overhead.

5. Motores DC — Leitura de Encoders

5.1 Biblioteca escolhida

Campo	Valor
Biblioteca	madhephaestus/ESP32Encoder
Base	Periférico PCNT (Pulse Counter) de hardware
Linguagem	C++
Dependências	Nenhuma

Repositório / Documentação: <https://github.com/madhephaestus/ESP32Encoder>

5.2 Justificativa

O ESP32 possui um periférico de hardware dedicado à contagem de pulsos chamado PCNT (Pulse Counter), que incrementa ou decrementa um registrador internamente sem consumir ciclos de CPU. A biblioteca ESP32Encoder é um wrapper que facilita o uso desse periférico.

Comparativo com interrupções por software:

Critério	Interrupção (software)	PCNT — ESP32Encoder
Carga na CPU	Alta	Nenhuma
Risco de perder pulsos	Sim, em alta velocidade	Não
Precisão	Depende do scheduler	Sempre precisa

5.3 Alternativas consideradas

paulstoffregen/Encoder: biblioteca clássica do Arduino, baseada em interrupções por software. Funciona no ESP32, mas com risco de perda de pulsos em altas velocidades.

Alternativa (paulstoffregen): <https://github.com/PaulStoffregen/Encoder>

Implementação manual com attachInterrupt() — possível, mas exige uso do atributo IRAM_ATTR e gestão manual de variáveis voláteis, com menor precisão.

6. Comunicação WebSocket — Telemetria em Tempo Real

6.1 Biblioteca escolhida

Campo	Valor
Biblioteca	links2004/WebSockets
Protocolo	WebSocket sobre Wi-Fi (IEEE 802.11)

Linguagem	C++
Dependências	WiFi.h (nativa do ESP32 Arduino)

Repositório / Documentação: <https://github.com/Links2004/arduinoWebSockets>

6.2 Justificativa

A telemetria em tempo real permite monitorar leituras dos sensores ToF, velocidade dos motores, contagem dos encoders e estado do algoritmo sem interromper o funcionamento do robô. O WebSocket oferece baixa latência e conexão persistente, adequado para a taxa de atualização necessária (~10 Hz).

Motivos da escolha:

- Mais usada com ESP32 no framework Arduino — ampla base de exemplos.
- Suporta modo servidor e cliente WebSocket.
- Boa estabilidade e manutenção ativa.
- Integra bem com tasks FreeRTOS rodando em paralelo no ESP32.
- Menor complexidade de integração em comparação com AsyncWebServer.

6.3 Alternativa considerada

A combinação me-no-dev/AsyncTCP + me-no-dev/ESPAsyncWebServer oferece WebSocket assíncrono — não bloqueia o loop principal e lida melhor com múltiplos clientes. Recomendada caso a carga de tasks FreeRTOS exija maior estabilidade na camada de rede.

Alternativa (AsyncWebServer): <https://github.com/me-no-dev/ESPAsyncWebServer>

7. Controle PID — Velocidade dos Motores

7.1 Abordagem escolhida

Campo	Valor
Biblioteca	br.ccini/ArduinoPID (ou implementação própria)
Algoritmo	PID discreto com anti-windup
Entradas	Pulsos do encoder (ESP32Encoder) → velocidade real
Saída	Duty cycle via LEDC

Referência ArduinoPID: <https://github.com/br-automation-com/Arduino-PID-Library>

7.2 Justificativa

O controle PID fecha o loop entre velocidade desejada (setpoint) e velocidade real medida pelos encoders. Para o Micromouse, o PID atua individualmente em cada motor para garantir trajetória reta e curvas precisas no labirinto.

8. Configuração do platformio.ini

A seguir, a configuração consolidada do arquivo platformio.ini para o projeto, reunindo todas as bibliotecas escolhidas:

```
[env:micromouse]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
```

```
lib_deps =
  pololu/VL53L0X
  ElectronicCats/MPU6050
  madhephaestus/ESP32Encoder
  links2004/WebSockets
```

```
build_flags =
  -DARDUINO_USB_MODE=1
  -DARDUINO_USB_CDC_ON_BOOT=1
```

```
monitor_speed = 115200
upload_speed = 921600
```

As flags ARDUINO_USB_MODE e ARDUINO_USB_CDC_ON_BOOT habilitam o Serial via USB nativo do ESP32-S3, necessário para debug sem conversor UART externo.

9. Resumo das Escolhas

Módulo	Biblioteca	Link
Sensores ToF	pololu/VL53L0X	https://github.com/pololu/vl53l0x-arduino
Sensor Inercial	ElectronicCats/MPU6050	https://github.com/ElectronicCats/mpu6050
Ponte H (L298N)	API nativa (LEDC)	https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html
Encoders	madhephaestus/ESP32Encoder	https://github.com/madhephaestus/ESP32Encoder
WebSocket	links2004/WebSockets	https://github.com/Links2004/arduinoWebSockets
Debug Serial	Serial nativo + ArduinoLog	https://github.com/thijse/Arduino-Log